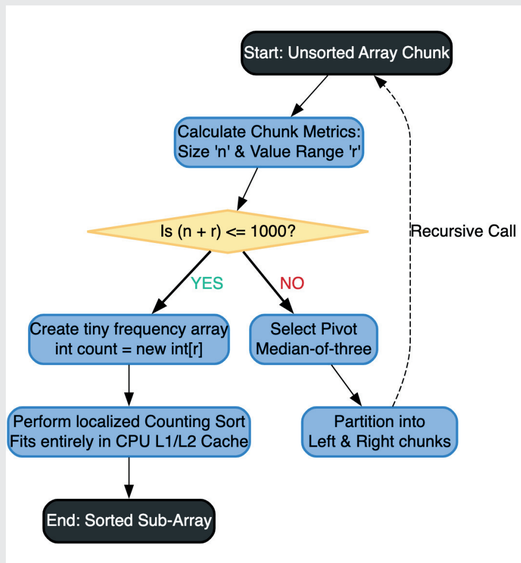
IMPROVING COUNTING SORT ALGORITHM VIA DATA LOCALITY

INTRODUCTION

Despite its impressive O(n+k) theoretical speed, Counting Sort's real-world performance collapses on sparse datasets because allocating massive, mostly empty frequency arrays triggers severe CPU cache misses. Replicating Paper 2, we investigate whether these hardware limitations should dictate algorithm design just as heavily as Big-O notation. Can preprocessing data to improve spatial locality bridge the gap between theoretical math and physical hardware?

HYBRID APPROACH TO DATA LOCALITY

We engineered a hybrid algorithm that merges comparison and non-comparison sorting. Instead of processing the entire dataset at once, a Quicksort phase recursively chops the data into localized blocks.



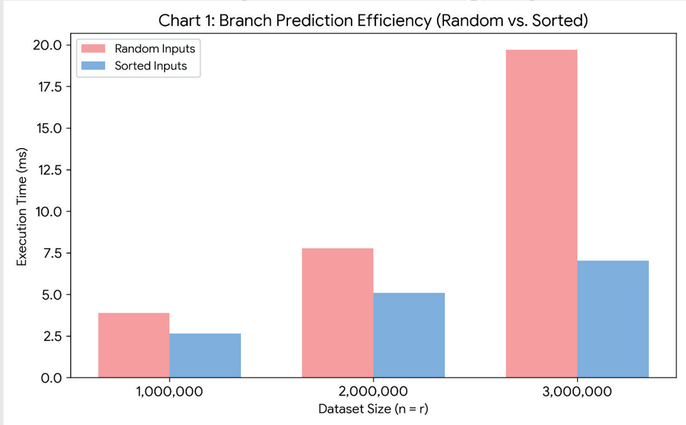
The algorithm stops chopping once a block's size and range hit a strict threshold r_chunk + n_chunk <= 1000. This guarantees the final Counting Sort uses a tiny frequency array perfectly sized for the CPU's ultra-fast L1 and L2 cache. But how does this cache-friendly design perform in physical speed tests

HOW DO WE TEST IT

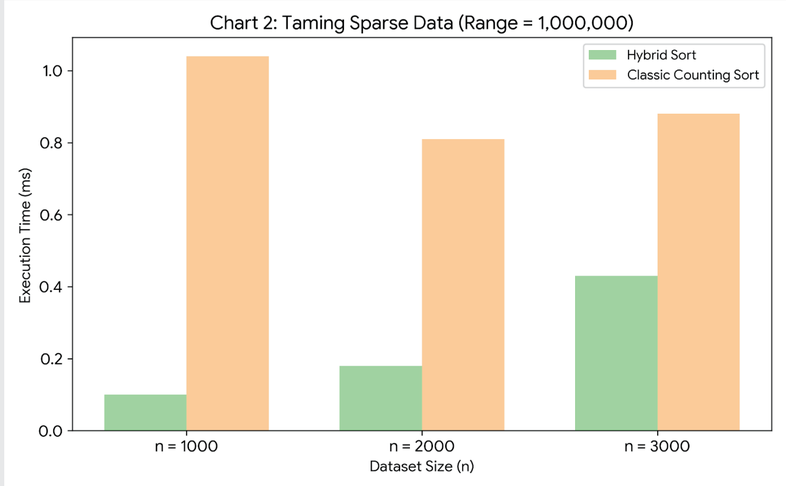
To find out, we raced our Hybrid algorithm against classic Counting Sort and Quicksort in a strict Java environment. To guarantee 100% fair speed tests, we forced memory cleanups to prevent background pauses (Garbage Collection) and "warmed up" the system so the computer couldn't cheat by memorizing data patterns.

EXPERIMENTS

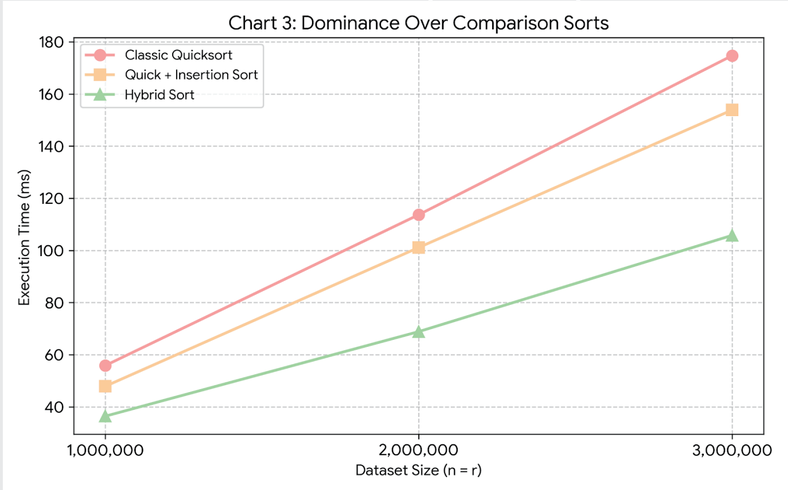
Our results clearly proved that memory organization dictates processing speed.



Sorting already-organized data was 3x faster than random data at 3,000,000 items, highlighting how random memory jumps waste valuable CPU time.



This hardware limit became obvious with sparse data (1,000 items across a 1,000,000 range). Classic Counting Sort choked at 1.04 ms, but our Hybrid algorithm finished in just 0.10 ms. Chopping the data into localized blocks completely avoided memory traffic jams, resulting in a massive 10x speedup.



At massive scales, our Hybrid model also beat classic Quicksort by about 40%. Stopping the chopping process early prevents the deep, memory-heavy operations that traditionally crash the computer.

CONCLUSION

TModern computers need data to be neatly packed in memory to run fast. By mixing the chopping power of Quicksort with the straight-line speed of Counting Sort, we successfully bypassed major hardware limits. In the future, programmers designing algorithms must pay just as much attention to physical computer memory as they do to mathematical speed.

Group 14

Dermot yang, Li Weihao, Robbie Devilly, Thehan aloka